

0.68.4. La fonction lire() — lire une case

`#include <lire>` : `lire(colonne, ligne)` rend la tuile posée dans cette case — de quoi savoir si elle est libre.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `lire(colonne, ligne)` rend le **numéro de la tuile** affichée dans cette case. Une case vide rend 0. C'est ainsi qu'un jeu sait s'il y a un mur devant le joueur, sans tenir un tableau à côté.

La ligne à écrire : `#include <lire>`.

Ses arguments : la colonne et la ligne.

Ce qu'elle coûte : environ **90 octets** la première fois.

Essaie : lis la case (4, 3), qui est vide : `t` vaut 0, et le programme écrit NON.

On retrouve lire dans 43 leçons

[0.69](#), [0.69.1](#), [0.69.2](#), [0.69.3](#), [0.82](#), [0.82.1](#), [0.83.2](#), [0.93.1](#), [0.93.2](#), [0.93.3](#), [0.93.4](#), [0.93.5](#), [0.93.6](#), [0.93.7](#), [0.93.8](#), [0.93.9](#), [0.94](#), [0.94.1](#), [0.95](#), [0.95.1](#), [0.99](#), [0.99.1](#), [0.100](#), [0.100.1](#), [0.101](#), [0.101.1](#), [0.102](#), [0.102.1](#), [0.103](#), [0.104](#), [0.105](#), [0.105.1](#), [0.106](#), [0.107](#), [0.107.1](#), [0.108](#), [31](#), [36](#), [112](#), [113](#), [115](#), [122](#), [132](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **335 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      52 o ← la console, à l'allumage – la mise en route de la console
main()         51 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   58 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         80 o ← CopierTuiles – 5 lettres sur 43
"OUI"          3 o ← texte() – un texte gravé
"NON"          3 o ← texte() – un texte gravé
```

Lettres gravées : AINOUI

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 2    22 o sur place, dans main() + EcrireTexte (10 o)
poser() × 1     6 o sur place, dans main()
lire() × 1      4 o sur place, dans main()
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <lire> : lire une case ----  
  
#include <poser>  
#include <ALPHABET>  
#include <texte>  
#include <lire>      // lire() : la tuile d'une case
```

2. UNE VARIABLE — T

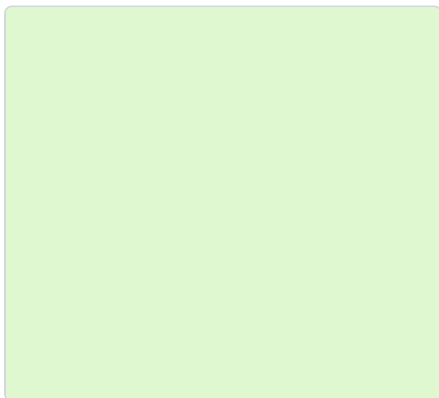
```
uint8_t t = 0;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

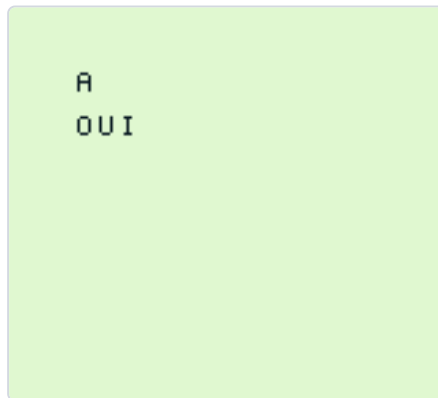
```
int main() {  
    poser(3, 3, ALPHABET[0]);    // un A en (3, 3)  
    t = lire(3, 3);              // t reçoit le numéro de la tuile : 1, celui du A  
  
    if (t == ALPHABET[0]) {      // est-ce un A ?  
        texte(3, 5, "OUI");  
    } else {  
        texte(3, 5, "NON");  
    }  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A, et OUI en dessous : la case lue porte bien un A.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ t vaut 1, le numéro du A (1)

✓ le programme a écrit OUI

Ensuite : **0.69. Chercher le A sur l'écran : lire** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++